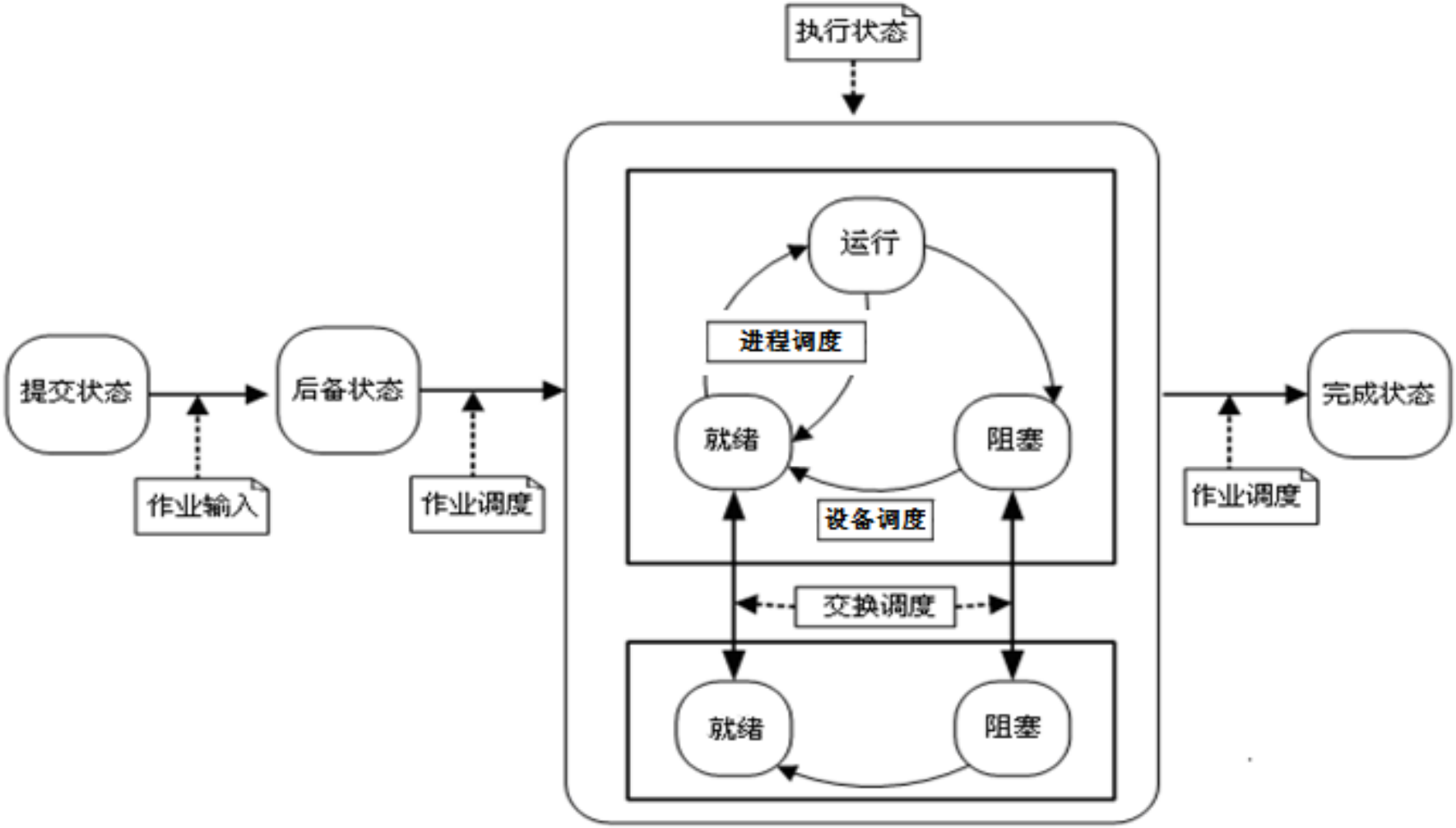
计算机操作系统

第4章 处理器调度(1)

调度

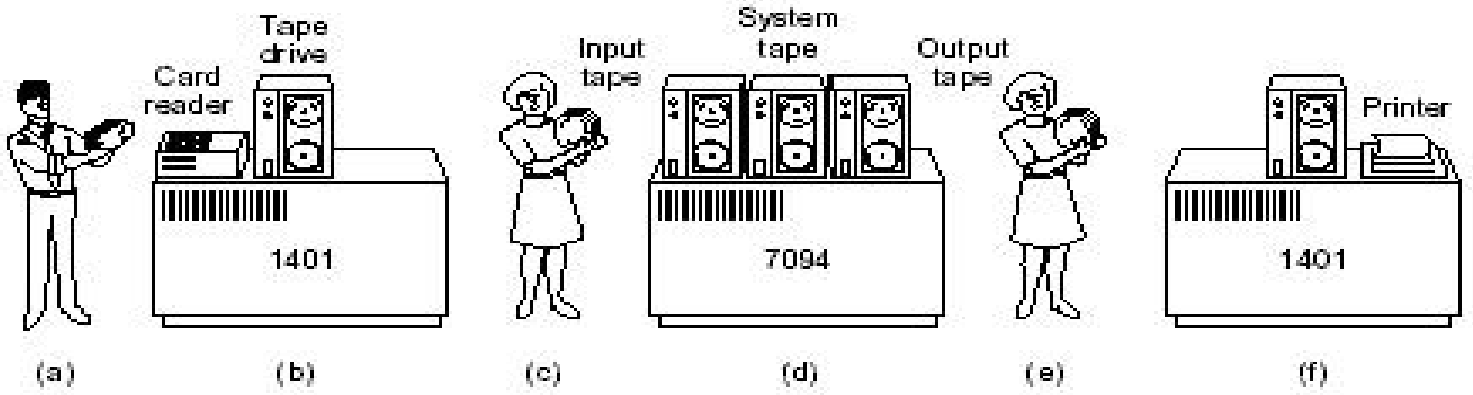
- 调度(Scheduling)是管理的一种方法，一种决策。其目的是通过一种合理的、有效的安排方式，提高资源（如工作、人力、车辆等）的利用率。
- 操作系统中关注的调度问题有四类：
  - 作业调度（宏观）——考虑何时执行哪些作业
  - 进程调度（微观）——考虑将哪个就绪进程转换为运行状态
  - 交换调度（中级）——考虑将进程如何在内/外存储器间交换
  - 设备调度——考虑让哪个进程（唤醒）使用设备

调度


执行状态
运行
进程调度
就绪
阻塞
设备调度
交换调度
就绪
阻塞
提交状态
后备状态
完成状态
作业输入
作业调度
作业调度

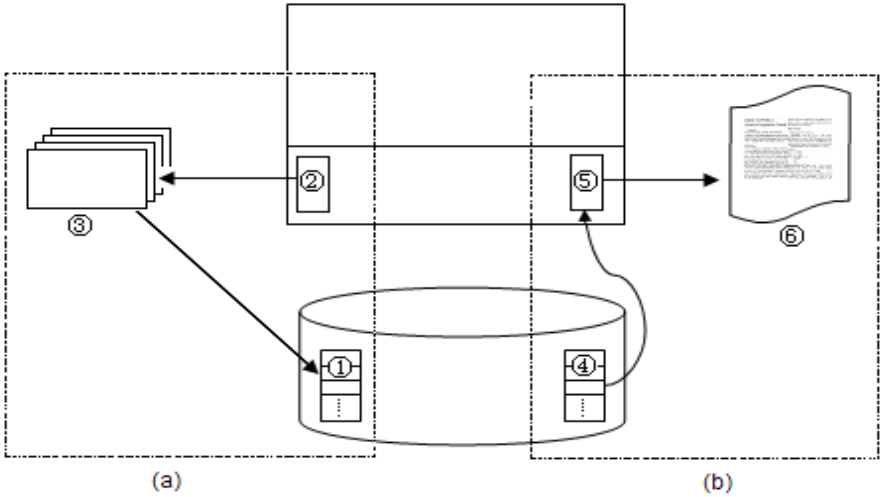
操作系统的基本类型-批处理系统

- 脱机批处理工作方式
  1. 提交：程序员将作业（程序、数据、作业说明书）提交给操作员
  2. 后备：操作员将一批性质相似的作业组成作业流，通过读卡机读入输入计算机，将这批作业信息转储到输入磁带上
  3. 执行：操作员将输入磁带放入主计算机，由主计算机操作系统调度这批作业的执行，期间无法干预。早期采用顺序执行方式（单道批处理），现在采用并发执行方式（多道批处理）。主计算机执行完毕后形成输出磁带。
  4. 完成：操作员将输出磁带放到输出计算机上进行输出。



操作系统的基本类型-批处理系统

- 联机批处理工作方式和SPOOLing技术（将脱机批处理的三部件集成）
  1. 预输入程序（图中2）：操作系统中控制读卡机、处理作业输入的程序，取代输入计算机
  2. 缓输出程序（图中5）：操作系统中控制输出设备、处理结果输出的程序，取代输出计算机
  3. 输入井（图中1）：磁盘上的特殊存储区域，取代输入磁带
  4. 输出井（图中4）：磁盘上的特殊存储区域，取代输出磁带



进程的动态性

- 进程的三种基本状态
  - 就绪：能够运行但却未在被处理器处理的状态
  - 运行：正在被处理器处理的状态
  - 阻塞：由于各种原因（IO操作、资源缺乏等）暂时无法运行的状态

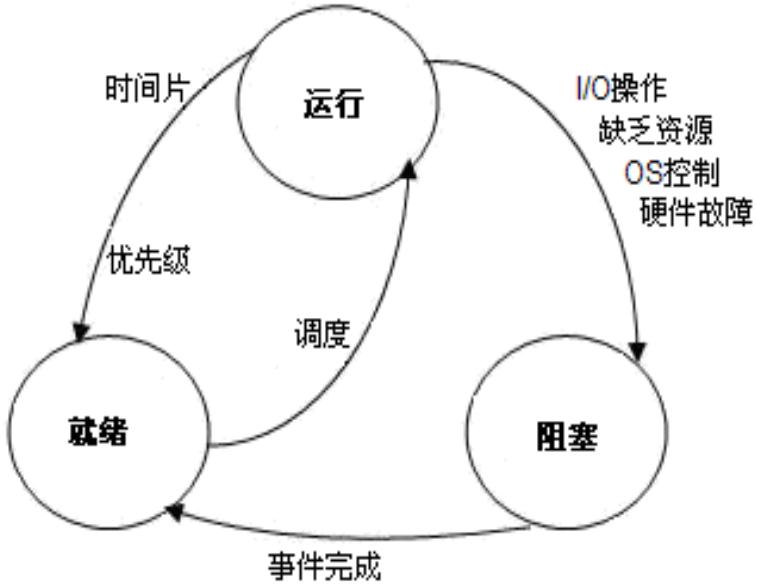
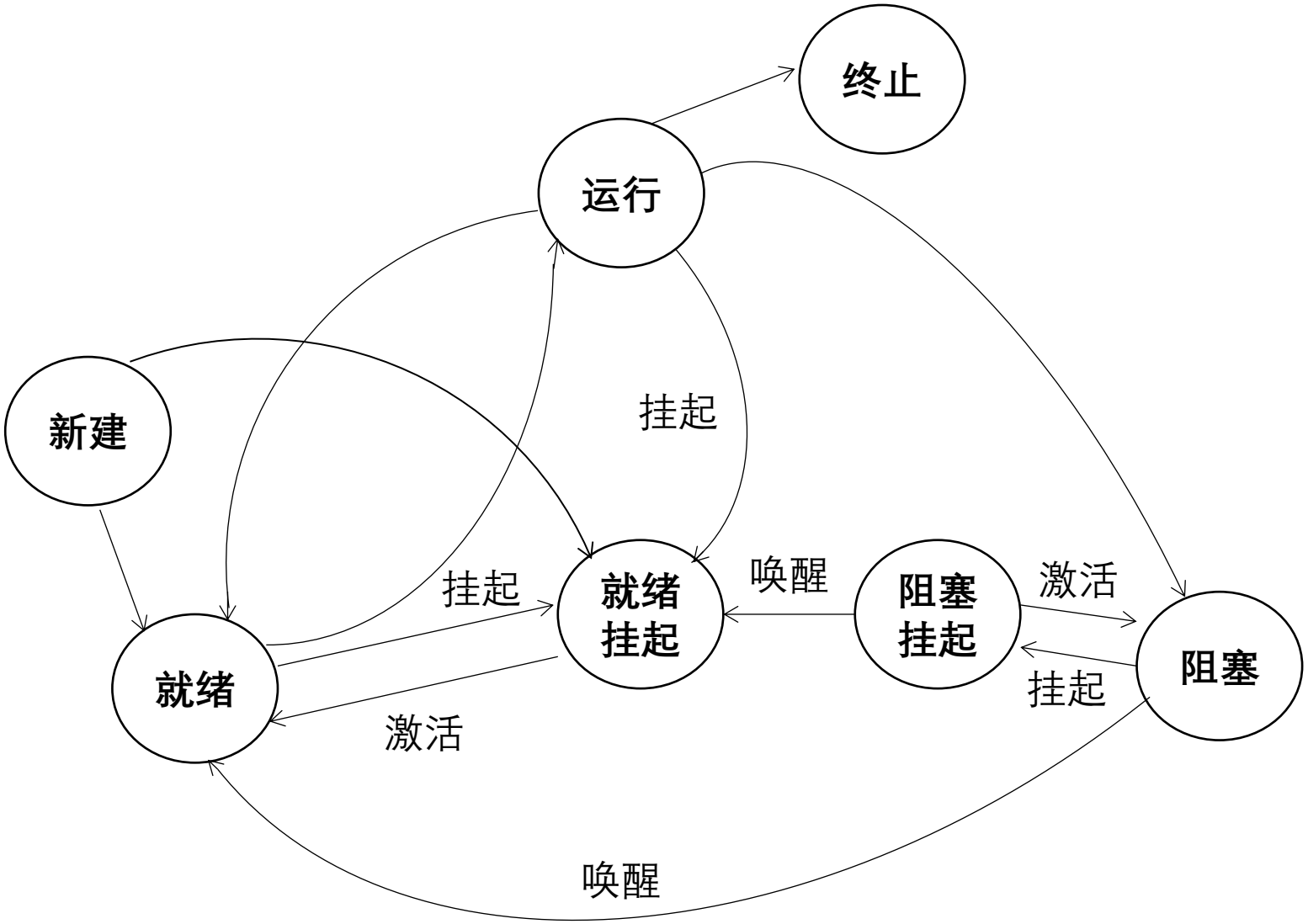


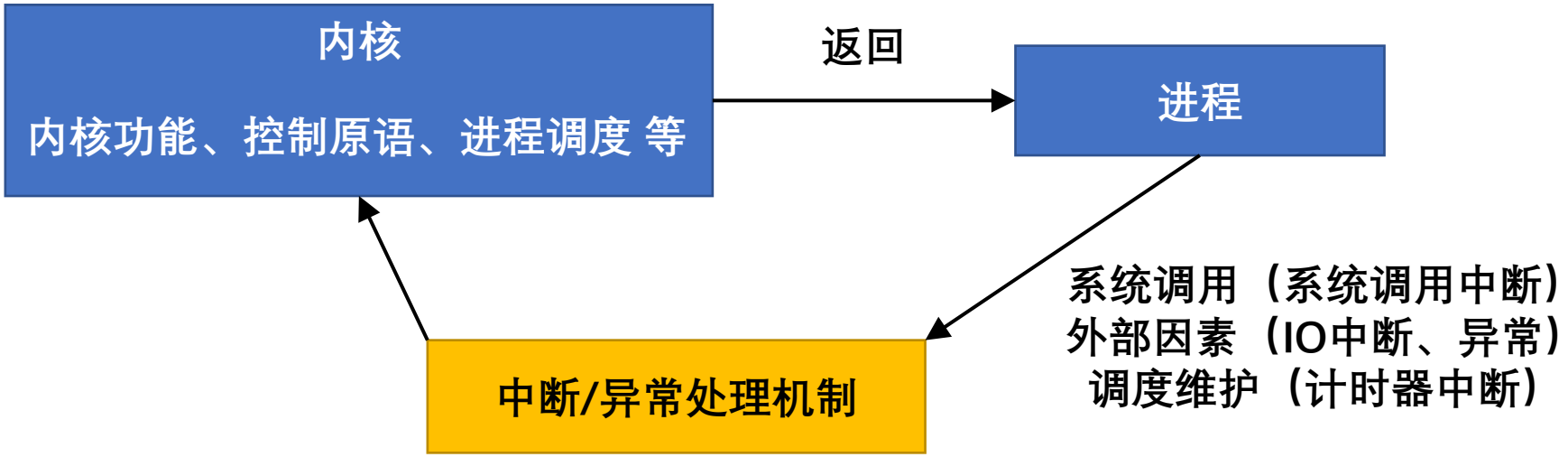
图3-1 进程状态转换关系图

七态---与挂起相关的状态转换(补充)



进程控制-切换

- 5. 进程切换使处理器的控制权在进程之间转换，无法通过操作系统内核独立完成，必须借助硬件来实现。



调度的性能指标

❖ 评价调度的效果，常使用一组性能指标：　　（批处理系统）

1. 周转时间和平均周转时间——从处理过程的完成时间进行评价

- 处理过程 J_r 的周转时间：$T_i = J_i$的完成时刻 $- J_i$的提交时刻
- 一批（n个）处理过程的平均周转时间：$T = (\sum_{i=0}^{n} T_i)/n$

注：周转时间并不直接体现某处理过程耗时的“合理性”

1. 带权周转时间和带权平均周转时间

- 处理过程 J_r 的带权周转时间：$T_{w_i} = T_i/T_{r_i}$
- 一批（n个）处理过程的平均带权周转时间：$T_w = \left(\sum_{i=0}^{n} T_{w_i}\right)/n$

注：T_{r_i}指作业运行时 J_r 占用处理器的总时间即作业 J_r的大小

调度的性能指标

- 评价调度的效果，常使用一组性能指标：(分时系统、实时系统)

  2. 响应时间——从处理过程的响应速度来进行评价处理过程J_i的响应时间：

     R_i = J_i第一次对用户做出响应的时刻 − J_i的提交时刻

  3. 其他性能指标

     - 公平合理——安排公平，用户满意
     - 资源利用率（并行度）——资源闲置的情况
     - 吞吐量——单位时间完成处理过程的数量

作业调度

- 作业状态
  - 提交状态
  - 后备状态
  - 执行状态
  - 完成状态
- 作业调度的功能
  - 设计数据结构，登记参数
  - 执行指定算法，从作业后备队列中选择一个作业
  - 为选中的作业分配资源，创建进程
  - 作业完成时的资源回收

作业调度算法(1)——FCFS

- 先到先服务（FCFS: First Come First Served）
  - 思想：总是选择最先提交(进入后备队列)的作业，即排队
  - 特点：
    - 公平合理
    - 算法简单，容易实现
    - 服务质量欠佳（不利于短作业，没有考虑任务紧迫性/重要性等）

作业调度算法(1)——FCFS

- 例子：假定在某单道批处理系统中，一批作业A、B、C和D依次先后到达，并同时等待执行。已知它们都是纯计算性的简单任务，运行时需要占用处理器的时间分别是10、3、2和5。把到达时间(提交时间)设为0。

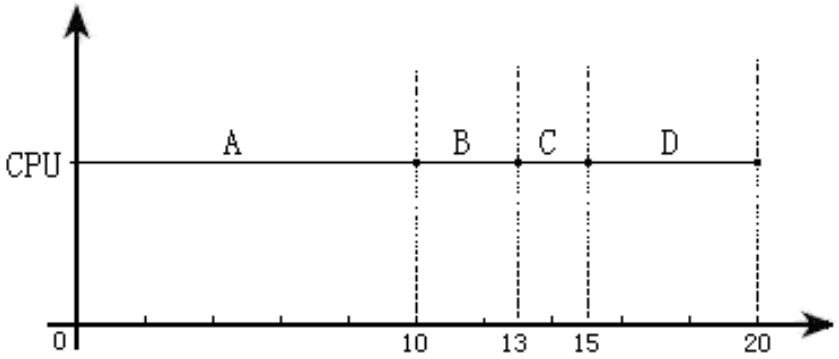


图4-2 FCFS算法例子调度图

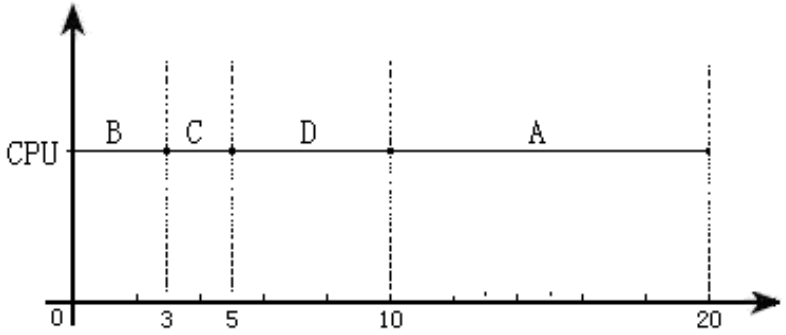


图4-3 非FCFS算法时的一种调度图

$$T = (T_A + T_B + T_C + T_D)/4$$
$$= (10+13+15+20)/4 = 14.5$$

$$T = (T_A + T_B + T_C + T_D)/4$$
$$= (20+3+5+10)/4 = 9.5$$

注意：实际的周转时间还应当考虑调度开始前已等待的时间，但在比较中不作讨论。

作业调度算法(2)——SJF

- 短作业优先（SJF: Shortest Job First）
  - 思想：总是选择需要占用处理器时间最小的作业，减少共同等待时间
  - 特点：
    - 拥有最小的平均周转时间
    - 吞吐量大
    - 存在“饥饿”现象（不利于长作业，没有考虑任务紧迫性/重要性等）
    - 算法思想简单，但实现困难（需要事先准确地估计作业的大小）

作业调度算法(2)——SJF

- 假定一批纯计算性的作业J_1、J_2、J_3、……、J_n，它们的作业大小及关系：$S_1 < S_2 < S_3 < \ldots < S_n$。不失一般性，假设这批作业提交时间都是0，每个作业运行时所需要的资源都能得到满足。
- 对于作业的任意排序$\Phi_0 = J_{i_1}J_{i_2}J_{i_3}\ldots J_{i_k}J_{i_{k+1}}\ldots J_{i_n}$，若将其中的相邻作业$J_{i_k}$和$J_{i_{k+1}}$交换位置使排序变为$\Phi_1 = J_{i_1}J_{i_2}J_{i_3}\ldots J_{i_{k+1}}J_{i_k}\ldots J_{i_n}$，称作对排序做一次临换。

$$J_{i_1}\ J_{i_2}\ J_{i_3}\ \ldots\ J_{i_k}\ J_{i_{k+1}}\ \ldots\ J_{i_n}$$



$$J_{i_1}\ J_{i_2}\ J_{i_3}\ \ldots\ J_{i_{k+1}}\ J_{i_k}\ \ldots\ J_{i_n}$$

作业调度算法(2)——SJF

- 用Γ_{before}和Γ_{after}分别表示Φ_0中作业J_{i_k}之前和作业$J_{i_{k+1}}$之后的作业的总周转时间，S_{before}表示作业J_{i_k}之前的作业总大小，则Φ_0和Φ_1的总周转时间

$$\Gamma_0 = \Gamma_{before} + \Gamma_{after} + 2S_{before} + 2S_{i_k} + S_{i_{k+1}}$$

$$\Gamma_1 = \Gamma_{before} + \Gamma_{after} + 2S_{before} + 2S_{i_{k+1}} + S_{i_k}$$

- 若$S_{i_{k+1}} < S_{i_k}$，则有：$\Gamma_0 > \Gamma_1$
- 对于任意排列Φ_0都可使用“冒泡排序法”生成排序

$$\Phi_{SFJ} = J_1J_2J_3\ldots\ldots J_n$$

- 即存在临换序列$\Phi_0\Phi_1\ldots\Phi_{SFJ}$，其中，对于任意的$\Phi_t$和$\Phi_{t+1}$有$\Gamma_t > \Gamma_{t+1}$
- 于是，Φ_0的平均周转时间 $T = \Gamma_0/n > \Gamma_{SFJ}/n = T^{SFJ}$。
- 由Φ_0的任意性，可知SFJ的平均周转时间T^{SFJ}是所有排序中最小的。

作业调度算法(3)——HRN

- 高响应比优先（HRN: Highest Response-ratio Next）
  - 思想：从后备队列中选择最大响应比的一个作业

$$\text{响应比}R = \frac{\text{作业等待时间}}{\text{作业大小}}$$

    - 其中，作业等待时间 = 系统当前时间 - 作业提交时刻
  - 特点：
    - 正比体现FCFS的思想
    - 反比体现SJF的思想
    - 通过考虑作业已等待的时间解决SJF的饥饿问题
    - 实现困难（需要事先准确地估计作业的大小）

作业调度算法——例一

- 假定在某脱机单道批处理系统中，有一批作业，它们的提交时刻和作业大小如下表所示，假定在10:00时开始调度，求分别采用FCFS、SJF、HRN作业调度算法时的调度顺序、各作业的周转时间、各算法的平均周转时间。

作业号	提交时刻	作业大小(分钟)
J_1	9:00	48
J_2	9:10	60
J_3	9:45	36
J_4	10:00	24

作业调度算法——例一

作业号	提交时刻	作业大小(分钟)
J_1	9:00	48
J_2	9:10	60
J_3	9:45	36
J_4	10:00	24

	9:00	9:10	9:45	10:00	10:48	11:48	12:24	12:48
提交	J_1	J_2	J_3	J_4				
开始执行				J_1	J_2	J_3	J_4	
完成执行					J_1	J_2	J_3	J_4

$$T = (108+158+159+168)/4 = 148.25 \text{（分钟）}$$

作业调度算法——例一

作业号	提交时刻	作业大小(分钟)
J_1	9:00	48
J_2	9:10	60
J_3	9:45	36
J_4	10:00	24

	9:00	9:10	9:45	10:00	10:24	11:00	11:48	12:48
提交	J_1	J_2	J_3	J_4				
开始执行				J_4	J_3	J_1	J_2	
完成执行					J_4	J_3	J_1	J_2

$$T = (168+218+75+24)/4 = 121.25 \text{（分钟）}$$

作业调度算法——例一

作业号	提交时刻	作业大小(分钟)
J_1	9:00	48
J_2	9:10	60
J_3	9:45	36
J_4	10:00	24

	9:00	9:10	9:45	10:00	10:48	11:12	11:48	12:48
提交	J_1	J_2	J_3	J_4				
开始执行				J_1 (1.25)	J_4 (2)	J_3 (2.42)	J_2	
完成执行					J_1	J_4	J_3	J_2

$$T = (108+218+123+72)/4 = 130.25 \text{（分钟）}$$

作业调度算法——例二

- 在某联机单道批处理系统中，有一批作业，它们的提交时刻和作业大小如下表所示。分别采用FCFS、SJF、HRN作业调度算法时的调度顺序、各作业的周转时间、各算法的平均周转时间。

作业号	提交时刻	作业大小(分钟)
J_1	9:00	48
J_2	9:10	60
J_3	9:45	36
J_4	10:00	24

注：联机批处理系统中，作业提交时即可开始作业调度。

作业调度算法——例二

作业号	提交时刻	作业大小(分钟)
J_1	9:00	48
J_2	9:10	60
J_3	9:45	36
J_4	10:00	24

	9:00	9:10	9:45	9:48	10:00	10:48	11:24	11:48
提交	J_1	J_2	J_3		J_4			
开始执行	J_1		J_2			J_3	J_4	
完成执行			J_1			J_2	J_3	J_4

$$T = (48+98+99+108)/4 = 88.25 \text{（分钟）}$$

作业调度算法——例二

作业号	提交时刻	作业大小(分钟)
J_1	9:00	48
J_2	9:10	60
J_3	9:45	36
J_4	10:00	24

	9:00	9:10	9:45	9:48	10:00	10:24	10:48	11:48
提交	J_1	J_2	J_3		J_4			
开始执行	J_1			J_3		J_4	J_2	
完成执行				J_1		J_3	J_4	J_2

$$T = (48+158+39+48)/4 = 73.25 \text{（分钟）}$$

作业调度算法——例二

作业号	提交时刻	作业大小(分钟)
J_1	9:00	48
J_2	9:10	60
J_3	9:45	36
J_4	10:00	24

	9:00	9:10	9:45	9:48	10:00	10:48	11:12	11:48
提交	J_1	J_2	J_3		J_4			
开始执行	J_1 (0)			J_2 (0.63)		J_4 (2)	J_3	
完成执行				J_1		J_2	J_4	J_3

$$T = (48+98+123+72)/4 = 85.25 \text{（分钟）}$$

进程调度

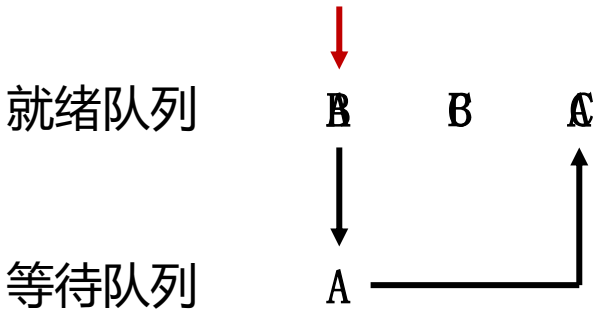
- 进程调度指在采用并发执行方式的系统中，决定正在运行的进程何时以何种方式将处理器使用权交给就绪队列中的哪个进程。
- 进程调度的主要功能：
  - 进程调度方式
    - 决定运行状态的进程何时以何种方式停止或暂停运行，让出处理器使用权
  - 进程调度算法
    - 从就绪队列中按照指定的算法选择一个进程，准备让其占用处理器执行。
  - 处理器切换
    - 系统要保存原来运行的进程（如果存在的话）的现场，为新选中的进程建立或恢复处理现场
  - 进程结束时资源回收
    - 进程结束后，进程调度程序回收进程占用的资源

进程调度方式

- 两种基本的进程调度方式：
  - 非抢占式
    - 除非运行的进程由于自身原因让出处理器使用权，否则一直运行直到结束。（非剥夺的）
    - 特点：管理简单、系统开销小，但是缺乏灵活性
  - 抢占式
    - 系统可以基于某些原则，在没有任何警告的情况下，让运行的进程停下来，把处理器分配给下一个进程。（剥夺式）
    - 特点：灵活，但是系统开销大

基本进程调度算法(1)——FCFS

- 先来先服务算法（FCFS: First Come First Served）
  - 方式：基于非抢占调度方式
  - 算法：总是选择最先进入就绪队列的进程（先进先出）
  - 特点：类似于FCFS作业调度。（公平，易实现，但服务质量欠佳）

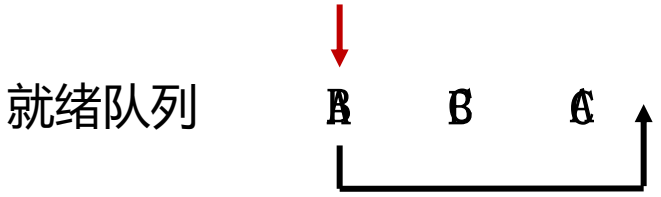


基本进程调度算法(2) ——SPN

- 短进程优先算法（SPN: Shortest Process Next）
  - 方式：基于非抢占调度方式
  - 算法：总是选择需要占用处理器时间最小的进程（剩余时间优先算法）
  - 特点：类似于SJF作业调度（平均周转时间最小，吞吐量大，但存在“饥饿”现象，实现困难）

基本进程调度算法(3)——RR

- 时间片轮转（RR: Round-Robin）
  - 方式：基于抢占调度方式，其实现依赖于计时器中断
  - 算法基本思想：
    - 总是选择最先进入就绪队列的进程（先进先出）
    - 进程每次被选中时分配一个时间片（可以不同）
    - 选中的进程在规定的时间片内运行（定时器）
      - ✓调度时判断其时间片是否用尽，若用尽，则将其移至就绪队列末尾，引起新的调度；否则（阻塞或结束），不修改就绪队列，并立即引起新的调度。



基本进程调度算法(3)——RR

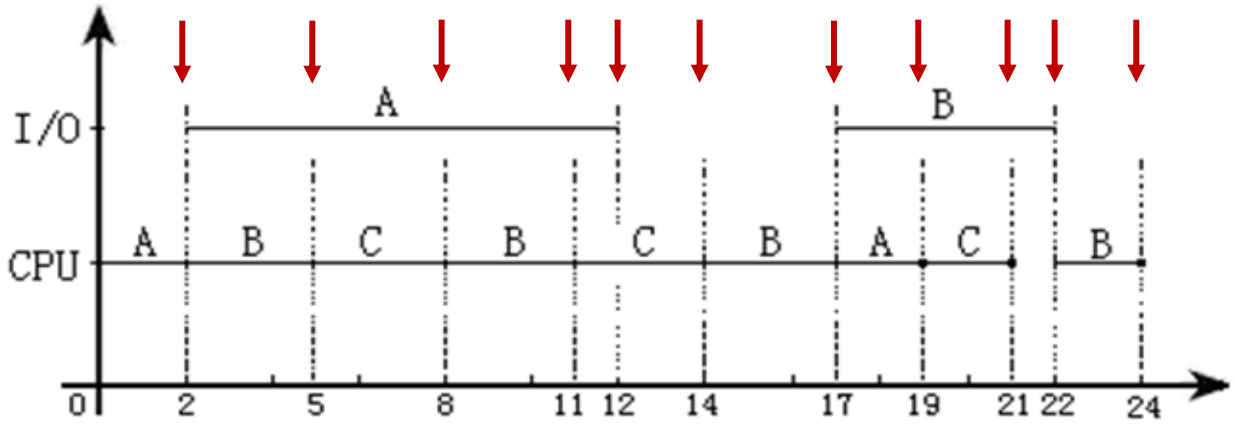
- 一些补充：
  - 时间片长度为T，若就绪队列中已存在n个进程，则提交的新进程的最长响应时间：R = T×n
  - 应用RR算法的关键是根据实际需要确定T的值
    - ✓T太长，响应时间过长，影响系统的及时性（若系统所有进程需要的处理器时间都小于系统的时间片，RR算法相当于FCFS算法。）
    - ✓T太短，进程切换过于频繁，系统开销加大

基本进程调度算法(3)——RR

- 例子：假定某分时系统有3个依次到达的进程A、B和C，它们的任务如下：

进程A		进程B		进程C	
2ms	CPU	9ms	CPU	8ms	CPU
10ms	I/O	5ms	I/O		
2ms	CPU	2ms	CPU		

  - 在采用简单RR算法，时间片为3ms时，请画出RR算法的调度图。



就绪队列 A B A C

等待队列 B

基本进程调度算法(4)——Priority

- 优先级算法（Priority）
  - 方式：可以基于抢占调度方式，也可以基于非抢占调度方式
    - 抢占调度方式，可保证总是执行高优先级进程
    - 非抢占调度方式，仅能保证当前一个进程阻塞或结束时总能选择优先级最高的进程
  - 算法：为每个进程赋予一个优先数，调度时总是选择优先数最高的进程。优先级算法设计的关键是：进程优先数配置：

基本进程调度算法(4)——Priority

- 根据不同的优先数配置策略，优先级算法可灵活地体现各种算法思想（FCFS、SJF等）。
- 根据不同的优先数配置特点，优先级算法分为：
  - ✓ 静态优先级算法：进程优先数在进程创建时确定，并不可改变。
    特点：思想简单，容易实现，但缺乏灵活性
  - ✓ 动态优先级算法：进程优先数在创建进程时被赋初值，运行中可根据系统状态改变。
    特点：适应性较好，但增加系统开销
- 优先数配置的考虑因素：
  - ✓ IO繁忙型进程应当被赋予较高优先级以提高系统并行度
  - ✓ 短进程应当被赋予较高优先级以提高系统吞吐量
  - ✓ 任务紧迫或重要的进程应当被赋予较高优先级，可以满足实时系统的要求

实时系统的进程调度

- 实时系统需要对周期性或非周期性发生的事件作出处理，其对处理正确性和时效性要求十分苛刻。
- 不同类型的事件对时效性的要求各异，可表达为以下时间参数：
  - 就绪时限：事件发生时到相应处理进程被创建的时间长度要求。
  - 开始时限：事件发生时到相应处理进程第一次运行的时间长度要求。
  - 完成时限：事件发生时到相应处理进程完成的时间长度要求。
  - 处理时间：某些特殊事件要求定期触发处理过程。

实时系统——可调度及其条件

- 对于一组事件或进程，如果存在某种处理方式，使其中每个事件的时限要求都可以得到满足，则称这组事件是可调度的（Schedulable）。否则，如果无论用何种方式，都至少有一个事件无法满足其时限要求，则称这组事件不可调度的。如果系统的所有事件或进程是可调度的，则称系统是可调度的。
- 可调度的必要条件：$\mu \geq \lambda$。
  - 其中μ表示系统单位时间内可以处理的事件数（处理能力），λ表示单位时间内到达的事件数。

实时系统——周期性事件的可调度

- 周期性事件是周期性触发的事件（处理时间），每次事件必须在下个周期到达前完成（完成时限）。
  - 若只有单一周期性事件，其发生周期为P，系统处理一次该事件需要的时间为C，则其可调度的充要条件：

$$P \geq C$$

    - $\mu \geq \lambda$； $\mu = 1/C$； $\lambda = 1/P$;
    - 从系统占用的角度来看，此充要条件为：$C/P \leq 1$
  - 若有n个周期性事件，发生周期分别为P_1、P_2、…、P_n，系统处理所需时间分别为C_1、C_2、…、C_n，则其可调度的充要条件：

$$\sum_{i=1}^{n} \frac{C_i}{P_i} \leq 1$$

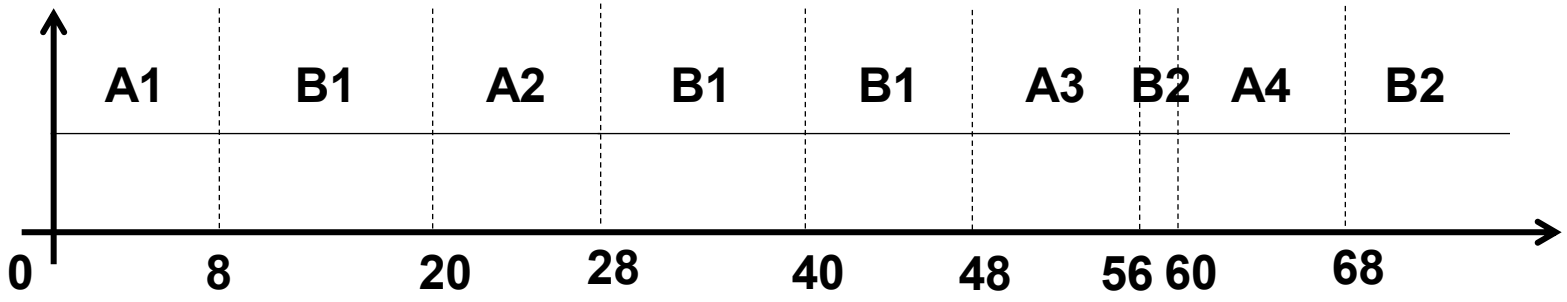
时限调度算法

- 时限调度算法
  - 是实时系统进程调度的常用算法，主要针对事件的完成时限要求
  - 基于抢占调度方式
  - 每次调度时选择完成时限最近（最迫切）的事件运行。
  - 如此调度，可使所有事件满足完成时限要求（若这组事件可调度）：
    - 每次选择时，若是可调度的，则可使当前最迫切的一个进程满足完成时限要求。
    - 每次选择不改变可调度性。

时限调度算法

- 例子：有2个周期性任务A、B，它们的周期分别是20ms和56ms，处理时间分别是8ms和32ms。画出采用时限调度算法的调度图。

就绪队列



调度-作业1

1. 某脱机单道批处理系统上，有三道作业 A、B、C 依次到达，到达时间及需要耗费的时间如下表所示。

	作业 A	作业 B	作业 C
提交时间	11:30	10:00	10:20
耗费时间	10 分钟	50 分钟	30 分钟

（1）若 12:00 开始调度，采用短作业优先算法。画出调度的时序图，计算平均周转时间和带权平均周转时间。

（2）若 12:00 开始调度，采用高响应比优先算法。画出调度的时序图，计算平均周转时间和带权平均周转时间。

调度-作业2

2. 进程 A、B、C 依次开始执行，各自的业务流程为：

进程 A		进程 B		进程 C	
CPU	2ms	CPU	10ms	CPU	15ms
IO	15ms	IO	5ms	IO	2ms
CPU	3ms	CPU	8ms	CPU	5ms

（假设三个进程所需操作的 IO 设备是同一个独占设备，依据 FCFS 原则分配）

（1）若采用先来先服务算法调度，画出运行时序图，并计算平均周转时间

（2）若采用时间片为 6ms 的时间片轮转调度，画出运行时序图，并计算平均周转时间。

调度-作业3

3.某实时系统采用时限调度算法来进行进程调度，运行时需要处理A、B、C三类周期性任务，其发生周期分别为10ms、22ms和36ms，单次任务耗时分别为4ms、10ms和8ms。则

1) 画出60ms前的调度图。（6分）

2) 这组任务是否可调度？请说明理由。（3分）

调度-测试题

4.一个多道批处理系统中仅有P1和P2两个作业，P2比P1晚 5ms到达。它们的计算和I/O 操作顺序如下：

P1：计算 60ms， I/O 80ms，计算 20ms

P2：计算 120ms， I/O 40ms，计算 40ms

若不考虑调度和切换时间，则完成两个作业需要的时间最少是（ ）。

A.240ms　　B.260ms　　C.340ms　　D.360ms

调度-测试题

已知：某进程调度程序采用基于优先数的调度策略，即选择优先数最大的进程运行，进程创建时由用户指定一个nice作为静态优先数。为了动态调整优先数，引入运行时间cpuTime和等待时间waitTime，初值均为0。进程处于执行态时，cpuTime定时加1，且waitTime置0；进程处于就绪态时，cpuTime置0，waitTime定时加1。请回答下列问题。

（1）若调度程序只将nice的值作为进程的优先数，即priority=nice，则可能会出现饥饿现象，为什么？

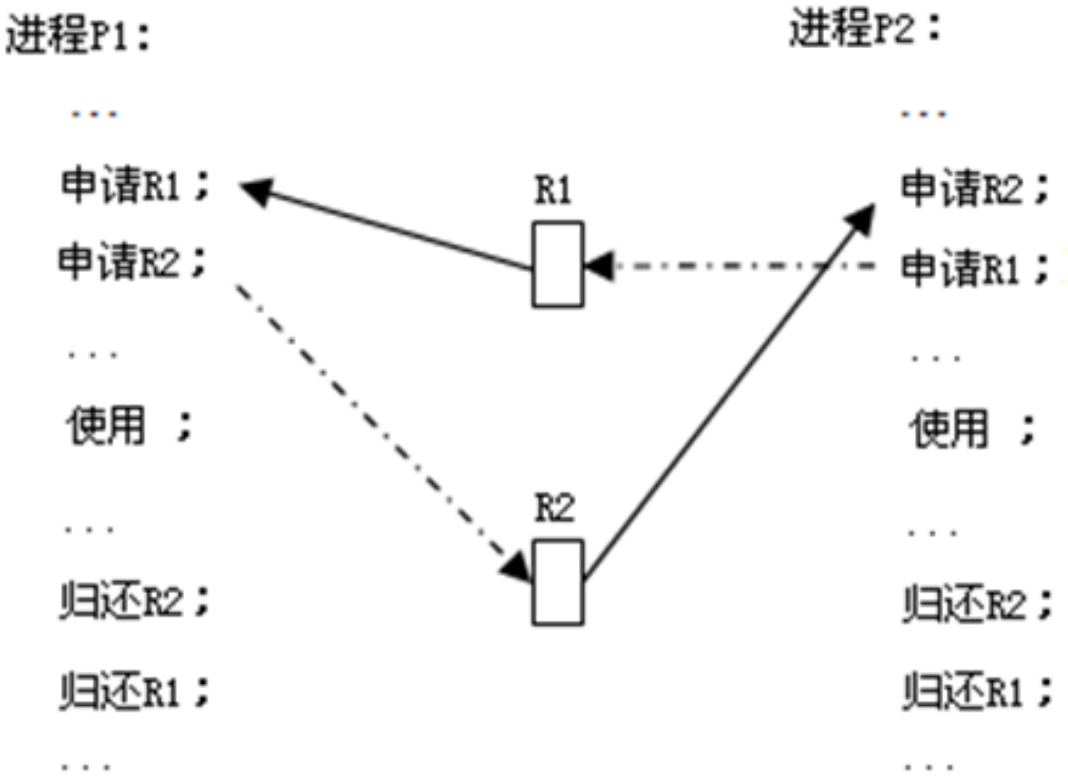
（2）使用nice、cpuTime和waitTime设计一种动态优先数计算方法，以避免产生饥饿现象，并说明waitTime的作用。

计算机操作系统

第4章 处理器调度(2)

死锁

- 一组进程均处于阻塞状态，其中的任一进程阻塞状态的解除都依赖于另一进程的后续操作，称这组进程处于死锁状态，简称死锁（Deadlock）。



死锁

- 死锁的分类：
  - 资源死锁——由于相互等待临界资源导致的死锁
  - 通信死锁——由于相互等待通信导致的死锁
  - 控制死锁——由于系统或用户的特殊控制导致的死锁
- 死锁的特点（与死循环区别）：
  - 偶发性——由特殊的并发执行顺序导致，是小概率事件
  - 非消耗——死锁的相关进程处于阻塞状态，不占用处理器
  - 程序无关——并非个别程序设计的错误，而是系统运行的错误

死锁

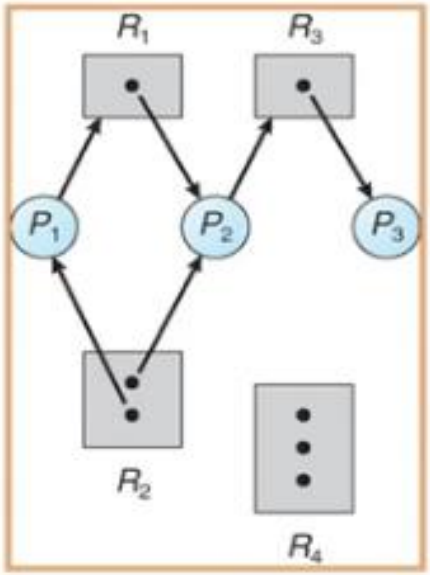
- （资源）死锁产生的根本原因是可用资源的数量小于所需资源的数量。但不可能靠无限制地增加资源数量来“根本地”解决死锁问题。
- 死锁发生的必要条件：
  - 互斥条件——死锁的相关进程是互斥的
  - 不剥夺条件——当进程申请资源时不能剥夺其他进程正在使用的资源
  - 请求与保持（部分分配）条件——进程在申请资源不成功时仍然保持已申请的资源
  - 环路等待条件——进程相互等待关系形成环路
- 死锁解决方法：预防 、避免、检测与恢复等三种

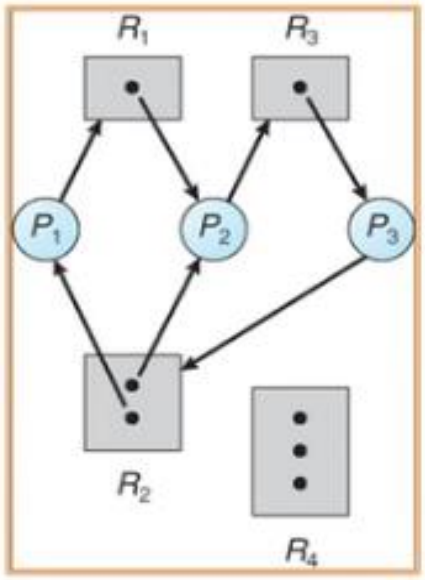
资源死锁

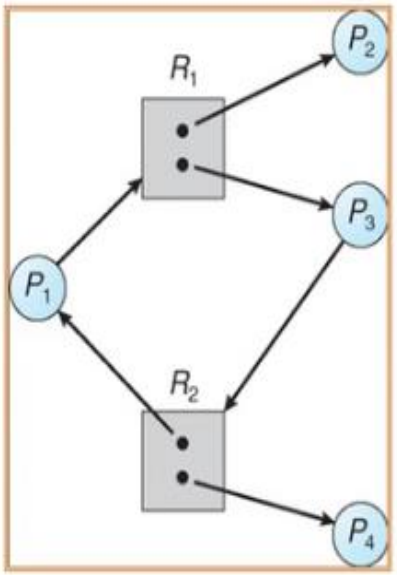
- 资源分配图G = (V，E)：表示当前资源分配与申请的情况。
  - V为结点集。V = P∪R，其中P是进程结点子集，R是资源结点子集
  - E为有向边集。从P中结点p_i指向R中结点r_j的边称作申请边，表示进程p_i申请一个单位的资源r_j，从R中结点r_j指向P中结点p_i的边称作分配边，表示已分配一个单位的资源r_i给进程p_j。
  - 在画图时，用圆“○”表示进程结点，用正方形“□”表示资源结点，如果某类资源的数量有多个，则在对应的正方形“□”内用实心圆点表示其数量。

资源死锁

- 死锁与环路
  - 死锁➔资源分配图中包含环路
  - 如果图中不包含环路➔没有死锁
  - 如果图中包含环路：
    - 如果每个资源类只有一个实例➔死锁
    - 如果每个资源类有几个实例➔可能死锁







死锁的预防

- 死锁的预防：在资源分配时采取一系列限制措施，破坏4条必要条件之一，以预防死锁的发生。
  - 1. “互斥条件”的破坏
    - 方法：不允许申请临界资源。
    - 缺陷：临界资源需要转化为非临界资源才可被申请和使用，大多数情况下无法实现。
  - 2. “不剥夺条件”的破坏（被动）
    - 方法：允许进程在申请一个资源得不到满足时，强制执行占有这个资源的进程的归还操作。
    - 缺陷：被剥夺资源的进程不可预期地被打断；系统需处理“回退”；反复剥夺与被剥夺。---无法体现公平性，出现不稳定、饥饿现象，增加开销。并且，进程申请新资源得不到满足时进入阻塞状态，并不一定意味着会产生死锁。

死锁的预防

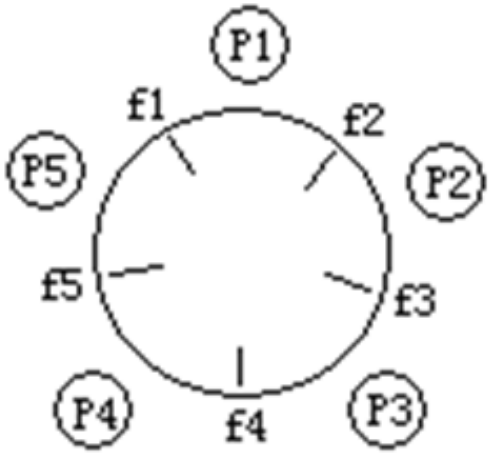
- 3. “请求与保持条件”的破坏
  - 方法1：静态分配——进程需要一次申请完所有可能使用的资源。
  - 缺陷：
    - ✓难以预知需要使用的资源；
    - ✓降低资源利用率。
  - 方法2：资源暂时释放（主动）——进程申请资源不成功时，暂时归还已分配的资源。
  - 缺陷：
    - ✓系统需处理“回退”；
    - ✓反复申请与归还。

死锁的预防

- 4. “环路等待条件”的破坏
  - 方法1：按序分配——对资源编号，进程只允许申请比当前已拥有资源编号更大的资源。
  - 缺陷：
    - ✓按照资源编号顺序申请使用资源，缺乏灵活性；
    - ✓编号管理困难；
    - ✓若使用大编号资源的段内需使用小编号资源，则只能实行小范围内的“静态分配”；
  - 方法2：单请求方式——进程必须把之前已经申请得到的资源全部归还后，才能申请新资源。
    - ✓缺陷：同时需要多个资源时只能实行小范围内的“静态分配”。

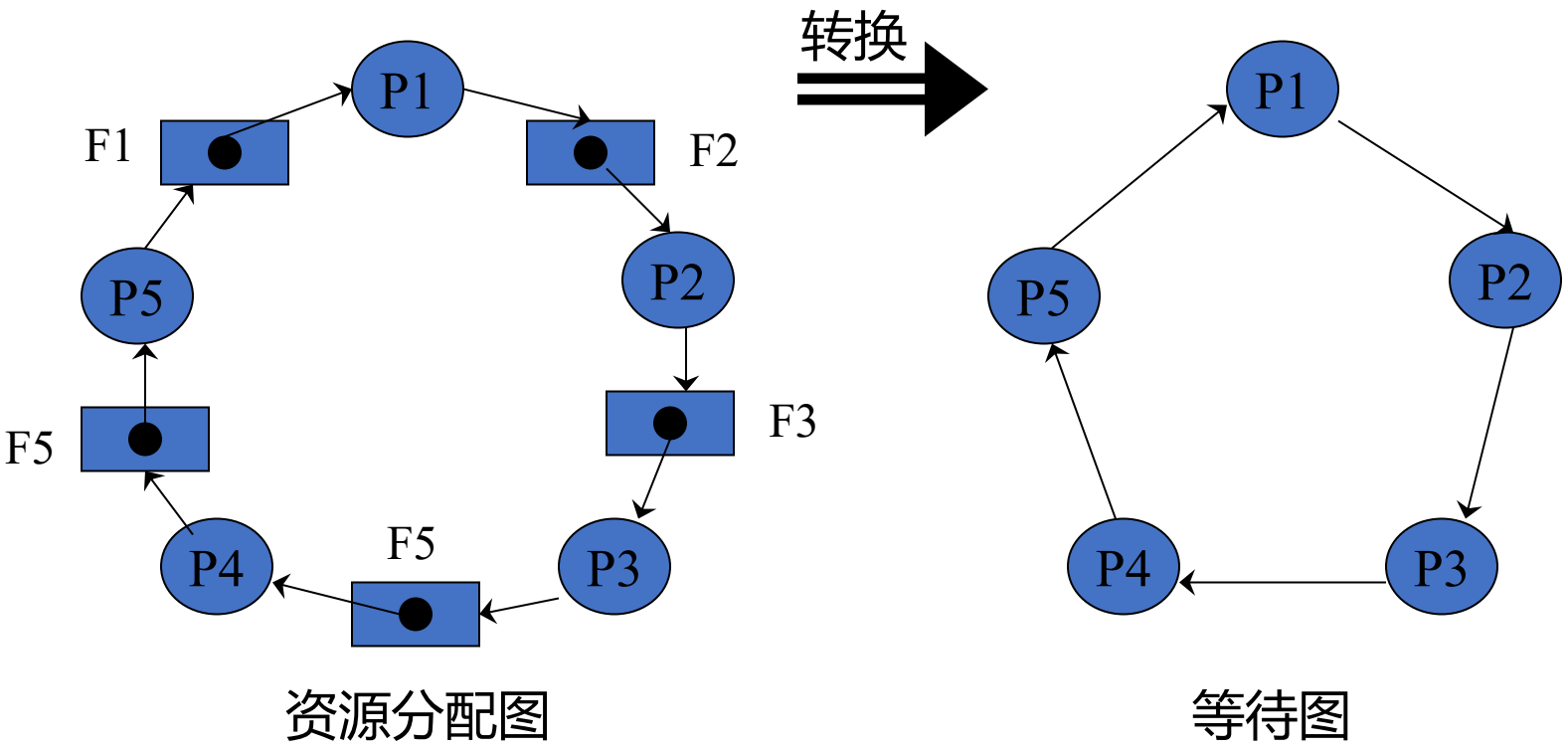
哲学家用餐问题

- 哲学家用餐问题：有5位哲学家围坐在一张圆桌周围共同讨论一问题。他们各自独立地或拿起筷子用餐或独自思考问题。假定桌上的每两位相邻的哲学家之间放一支筷子，每位哲学家在用餐时需要得到左右两边的筷子，然后才能用餐，用餐后放下筷子，又开始独立思考问题。如此反复。由于两位哲学家共享他们之间的一支筷子，哲学家们用餐和思考问题又具有随机性，那么，如何用信号量机制实现5个哲学家进程的并发执行。



哲学家用餐问题

- 哲学家用餐问题：当一个资源类型只有一个实例时，可以用更简单的等待图表示。



资源分配图　　等待图

哲学家用餐问题

❖ 哲学家用餐问题：并发程序设计

P1( )	P2( )	P3( )	P4( )	P5( )
{	{	{	{	{
思考;	思考;	思考;	思考;	思考;
p(s1);	p(s2);	p(s3);	p(s4);	p(s5);
拿f1;	拿f2;	拿f3;	拿f4;	拿f5;
p(s2);	p(s3);	p(s4);	p(s5);	p(s1);
拿f2;	拿f3;	拿f4;	拿f5;	拿f1;
用餐;	用餐;	用餐;	用餐;	用餐;
放下f2;	放下f3;	放下f4;	放下f5;	放下f1;
v(s2);	v(s3);	v(s4);	v(s5);	v(s1);
放下f1;	放下f2;	放下f3;	放下f4;	放下f5;
v(s1);	v(s2);	v(s3);	v(s4);	v(s5);
}	}	}	}	}

死锁的避免

- 死锁的避免：允许进程动态地申请资源，但系统在资源分配时进行系统的安全状态检查，如果分配后系统处于安全状态，则实施分配，否则就不分配。
- 死锁避免的特点（与死锁的预防相区别）：
  - 原理上是通过预测潜在的死锁
  - 不影响程序设计
  - 临界资源分配并非“空闲让进”

资源分配状态

- n个并发进程$P_1 \sim P_n$，m种资源$R_1 \sim R_m$，考虑某时刻的资源分配状态：
  - 当前可用向量Available，其中Available[j]表示当前空闲的资源R_j的数量
  - 当前分配矩阵Used，其中Used[i, j]表示P_i已获得R_j的数量
  - 最大需求矩阵Max，其中Max[i, j]表示P_i对R_j的最大需求量

安全状态和安全序列

- 系统在已知一组进程对所有资源的最大需求量和当前分配量的基础上得出当前资源分配状态，所谓安全状态是指：可以找出一个处理器执行这些进程的顺序，按照这个顺序依次执行各个进程，每个进程都可以运行完成。
- 把处理器执行这些进程的顺序称为进程的安全序列。

Process	Max	Used
A	8	3
B	3	1
C	10	2
D	7	4

当前资源R的可用数：2

安全状态

安全序列：B D A C

安全状态检查

- 检查某个状态是否是安全状态的思路：“最坏情况”下的模拟运行
  - 假设各进程立即申请所有未满足的资源，且在其执行结束前不归还
  - 试图找出安全序列
- 模拟运行的临时参数：
  - 矩阵Need表示各进程未满足的资源数量，Need = Max – Used
  - 向量SimAvailable记录模拟运行中的资源可用量，初始值为Available
  - 向量Finished记录模拟运行的进程顺序，初始值为0

安全状态检查

- 模拟运行的步骤：
  - 1. 每次选取满足以下条件的进程P_s进行模拟运行

    Finished[s]=0 且 对所有j，Need[s, j] ≤ SimAvailable[j]

    若不存在这样的进程，则转步骤3
  - 2. 选中的P_s模拟运行完毕，则将归还所占用的资源：

    对所有j，SimAvailable[j] = SimAvailable[j] + Used[s, j]

    同时记录P_s的模拟运行序号Finished[s] = ++index

    然后跳转到步骤1
  - 3. 得出结论：
    - 若向量Finished中存在0项，则原状态不是安全状态
    - 若向量Finished中不存在0项，则原状态是安全状态，向量Finished指出了其安全序列。

安全状态检查—实例

- 例：某系统有4类资源A、B、C、D，数量分别为8、10、9、12。当前有5个进程P1、P2、P3、P4、P5，最大需求矩阵Max和当前分配矩阵Used如下图所示。

$$Max=\begin{bmatrix}4&6&3&8\\3&3&5&2\\6&6&0&9\\3&4&8&7\\4&3&2&5\end{bmatrix}\qquad Used=\begin{bmatrix}1&1&0&1\\2&2&4&0\\0&5&0&1\\1&1&1&5\\2&0&2&2\end{bmatrix}$$

  - 当前系统是否为安全状态?

安全状态检查—实例

当前系统是否为安全状态?

资源总量 (8, 10, 9,12)

进程	Max				Used				Need				Available				Finished
	A	B	C	D	A	B	C	D	A	B	C	D	2	1	2	2	
P_1	4	6	3	8	1	1	0	1	3	5	3	7	8	10	9	12	5
P_2	3	3	5	2	2	2	4	1	1	1	1	1	4	3	6	3	1
P_3	6	6	0	9	0	5	0	1	6	1	0	8	7	9	9	11	4
P_4	3	4	8	7	1	1	1	5	2	3	7	2	7	4	9	10	3
P_5	4	3	2	5	2	0	2	2	2	3	0	3	6	3	8	5	2

银行家算法

- 如果系统当前处于安全状态，则仍有可能存在进程死锁，只有在之后的每次资源分配后都能保证系统处于安全状态，才能保证不存在进程死锁；
- 如果系统当前处于不安全状态，则系统中一定存在一组进程，它们将进入死锁状态；

Process	Max	Used
A	8	3
B	3	1
C	10	2
D	7	4

当前资源R的可用数：2

安全状态

安全序列：B D A C

错误的
资源分配

Process	Max	Used
A	8	3
B	3	1
C	10	2
D	7	5

当前资源R的可用数：1

非安全状态

银行家算法

- 目的：对每次资源申请都判断可否分配以保证系统总在安全状态。
- 系统环境：n个并发进程$P_1 \sim P_n$，m种资源$R_1 \sim R_m$
- 对于某个进程P_t发出的某次资源申请，算法输入：
  - 系统总资源向量Total，其中Total[j]表示R_j的总量
  - 最大需求矩阵Max，其中Max[i, j]表示P_i对R_j的最大需求量
  - 当前分配矩阵Used，其中Used[i, j]表示P_i已获得R_j的数量
  - 申请向量Request，其中Request[j]表示P_t对R_j的申请数量
- 算法输出： P_t的资源申请Request能否得到分配。

银行家算法

- 算法步骤：
  - 1. 初始化，计算资源可用向量Available和当前需求矩阵Need

    对所有j，$Available[j] = Total[j] - \sum_i Used[i, j]$

    $Need = Max - Used$
  - 2. 合法检查，检查资源申请与需求矩阵的相容性，未通过则报错

    对所有j，检查：$Request[j] \le Need[t, j]$？
  - 3. 资源检查，检查请求的资源是否可用，未通过则阻塞进程Pt

    对所有j，检查：$Request[j] \le Available[j]$？
  - 4. 预分配，假定Request得到满足

    $Available = Available - Request$

    $Used[t, j] = Used[t, j] + Request[j]$

    $Need[t, j] = Need[t, j] - Request[j]$
  - 5. 安全状态检查，检查预分配后的状态是否为安全状态，若不是安全状态则取消第4步的预分配，并阻塞进程Pt。

银行家算法—实例

- 例：某系统有4类资源A、B、C、D，数量分别为8、10、9、12。当前有5个进程P1、P2、P3、P4、P5，最大需求矩阵Max和当前分配矩阵Used如下图所示。

$$Max=\begin{bmatrix}4&6&3&8\\3&3&5&2\\6&6&0&9\\3&4&8&7\\4&3&2&5\end{bmatrix}\qquad Used=\begin{bmatrix}1&1&0&1\\2&2&4&0\\0&5&0&1\\1&1&1&5\\2&0&2&2\end{bmatrix}$$

  - 1) 在当前状态下，如果进程P1申请request=(1，0，1，0)，系统能否分配?
  - 2) 在当前状态下，如果进程P3申请request=(1，0，0，1)，系统能否分配?

银行家算法—实例

- 1) 在当前状态下，如果进程P1申请request=(1，0，1，0)，系统能否分配?
- 资源总量 (8, 10, 9,12)

进程	Max				Used				Need				Available				Finished
	A	B	C	D	A	B	C	D	A	B	C	D	1	1	1	2	
P_1	4	6	3	8	2	1	1	1	2	5	2	7	8	10	9	12	5
P_2	3	3	5	2	2	2	4	1	1	1	1	1	3	3	5	3	1
P_3	6	6	0	9	0	5	0	1	6	1	0	8	6	9	8	11	4
P_4	3	4	8	7	1	1	1	5	2	3	7	2	6	4	8	10	3
P_5	4	3	2	5	2	0	2	2	2	3	0	3	5	3	7	5	2

银行家算法—实例

- 2) 在当前状态下，如果进程P3申请request=(1，0，0，1)，系统能否分配?
- 资源总量 (8, 10, 9,12)

进程	Max				Used				Need				Available				Finished
	A	B	C	D	A	B	C	D	A	B	C	D	1	1	2	1	
P_1	4	6	3	8	1	1	0	1	3	5	3	7					
P_2	3	3	5	2	2	2	4	1	1	1	1	1	3	3	6	2	1
P_3	6	6	0	9	1	5	0	2	5	1	0	7					
P_4	3	4	8	7	1	1	1	5	2	3	7	2					
P_5	4	3	2	5	2	0	2	2	2	3	0	3					

死锁的检测与恢复

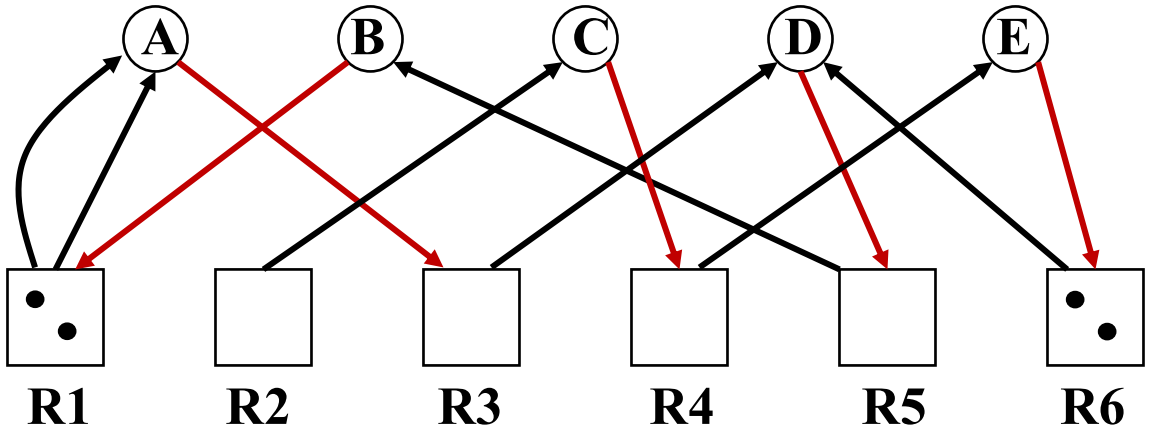
- 若系统未采取任何预防死锁或避免死锁的措施，允许系统存在进程死锁，则可以安排专门的检测程序，定期对系统进行检查，若发现某些进程处于死锁，将其恢复。
- 死锁检测的思路：“最好情况”下的模拟运行
  - 假设各进程在满足了当前的资源申请后将立即归还已分配资源
  - 试图找到满足所有进程当前资源申请的分配顺序
- 在死锁检测中，用资源分配图G = (V，E)表示当前资源分配与申请的情况。

死锁的检测与恢复

- 例：系统拥有的资源有R1、R2、R3、R4、R5和R6数量分别为2、1、1、1、1和2，当前进程有A、B、C、D和E，已知当前进程和资源的申请、分配关系，如下表所示。请画出当前系统的资源分配图，并给出简化过程。

进程	已分配的资源	申请的资源
A	2个单位的R1	1个单位的R3
B	1个单位的R5	1个单位的R1
C	1个单位的R2	1个单位的R4
D	1个单位的R3，1个单位的R6	1个单位的R5
E	1个单位的R4	1个单位的R6

死锁的检测与恢复



进程A、B、D死锁

- 利用资源分配图的化简来表达“模拟运行”过程，从而检测死锁
  - 1. 找到满足以下条件的进程P_s，若找不到则跳转到步骤3

    P_s不是孤立点，且P_s的所有申请边都可得到分配（或无申请边）
  - 2. 模拟P_s获得分配后归还已分配资源，即消除其所有申请边与分配边，回到步骤1
  - 3. 得出结论：若图中全为孤立点则当前未死锁，否则当前死锁，非孤立点表示死锁的进程

死锁的检测与恢复

- 死锁的恢复主要有以下三种方法：
  - 剥夺资源：在死锁发生时（与预防阶段不同），剥夺（某个）死锁进程的资源。
  - 撤销进程：在死锁发生时撤销（某个）死锁的进程。
  - 重新启动系统：在死锁发生时重新执行所有进程。

影响小

影响大

- 鸵鸟算法：由于死锁现象是小概率事件，现有的操作系统一般不考虑它的存在，而不做特殊应对。

死锁-作业1

2. 某系统采用银行家算法避免死锁。系统中有三类临界资源 A、B 和 C，总数分别是 5、11 和 11。当前有 5 个进程 P1、P2、P3、P4 和 P5，已知最大需求矩阵 Max 和当前分配矩阵

MAX

	A	B	C
P1	3	5	11
P2	1	6	6
P3	0	2	3
P4	2	8	5
P5	4	4	5

USED

	A	B	C
P1	0	2	3
P2	1	3	3
P3	0	1	2
P4	1	1	1
P5	3	2	0

（1）当前系统是否是安全状态？如果是，请写出安全序列。
（2）若在此状态下进程 P2 申请（0，1，1）能否分配？
（3）若在此状态下进程 P5 申请（0，1，0）能否分配？

死锁-作业2

得分	评卷人

1. 系统有四个进程 A、B、C、D 共享临界资源 R1 和 R2。已知 R1、R2 的数量各是 12 和 8，进程的最大需求量 Max 和当前已分配矩阵 Used 如图 2 所示，问：当前系统是否处于安全状态？并说明理由。

Process	Max		Used	
	R1	R2	R1	R2
A	8	2	3	1
B	3	3	1	2
C	10	2	2	1
D	7	6	4	3

图2

死锁-作业3

五、 某系统采用银行家算法避免死锁。系统中有三类临界资源 A、B 和 C，总数分别是 10、12 和 13。当前有 5 个进程 P1、P2、P3、P4 和 P5，已知最大需求矩阵 Max 和当前分配矩阵 Used 如下：

$$Max = \begin{pmatrix} 9 & 3 & 5 \\ 5 & 4 & 5 \\ 7 & 10 & 2 \\ 3 & 2 & 4 \\ 4 & 8 & 7 \end{pmatrix} \qquad Used = \begin{pmatrix} 3 & 2 & 1 \\ 2 & 2 & 2 \\ 1 & 5 & 2 \\ 0 & 0 & 4 \\ 0 & 1 & 3 \end{pmatrix}$$

（1）当前系统是否是安全状态？（3 分）

（2）若在此状态下进程 P2 申请（1，0，1）能否分配？（4 分）

（3）若在此状态下进程 P5 申请（1，0，0）能否分配？（4 分）

死锁-测试题1

3.设系统中有3种类型的资源（A，B，C）和5个进程（P1，P2，P3，P4，P5），A资源的数量为17，B资源的数量为5，C资源的数量为20。在T0时刻系统状态如下表：
系统采用银行家算法实施死锁避免策略。
（1）T0时刻是否为安全状态（请写出分析过程）？若是，请给出安全序列。
（2）若进程P4请求资源（2，0，1），是否能实施资源分配？为什么？

进程	最大资源需求量			已分配资源数量		
	A	B	C	A	B	C
P1	5	5	9	2	1	2
P2	5	3	6	4	0	2
P3	4	0	11	4	0	5
P4	4	2	5	2	0	4
P5	4	2	4	3	1	4

死锁-测试题2

2. 哲学家就餐问题中，如果将必须先拿起左边的筷子的哲学家称为左撇子，而将必须先拿起右边的筷子的哲学家称为右撇子。在同时存在左撇子和右撇子的前提下，让哲学家随意就座。请问此时会产生死锁吗？为什么？